



Reinforcement Learning

Avec Python (inspiré par Patrick Loeber)

Reinforcement Learning

- L'apprentissage par renforcement (RL) est un domaine de l'apprentissage automatique qui s'intéresse à la manière dont les agents logiciels devraient prendre des actions dans un environnement afin de maximiser la notion de récompense cumulative.
- Autrement dit, le RL est le fait d'enseigner à un agent logiciel comment se comporter dans un environnement en lui disant dans quelle mesure il réussit bien.
- Librement inspiré de : Patrick Loeber
 - **GH**
 - <https://github.com/patrickloeber/snake-ai-pytorch>
 - **YTB**
 - <https://www.youtube.com/watch?v=PJI4iabBEz0&list=PLqnsIRFeH2UrDh7vUmJ60YrmWd64mTTKV>

Deep Q Learning

- Cette approche étend l'apprentissage par renforcement en utilisant un réseau neuronal profond pour prédire les actions.

Il faudra implémenter 3 blocs

- **Agent**
 - **game**
 - **model**
 - **Training:**
 - state = get_state(game)
 - action = get_move(state):
 - model.predict()
 - reward, game_over, score = game.play_step(action)
 - new_state = get_state(game)
 - remember
 - model.train()
- **Game (PyGame)**
 - play_step(action)
 - -> reward, game_over, score
- **Model (Torch)**
 - **Linear_QNet (DQN)**
 - model.predict(state)
 - -> action

Systeme de récompenses (Reward) & Action

Reward

- Attraper la pomme : +10
- Perdre : -10
- Autres actions : 0
- Se déplacer : 0,1
- Finir le jeu : 100

Action

- HAUT : (0, -1)
- BAS : (0, 1)
- GAUCHE : (-1, 0)
- DROITE : (1, 0)

Les états (States)

Danger

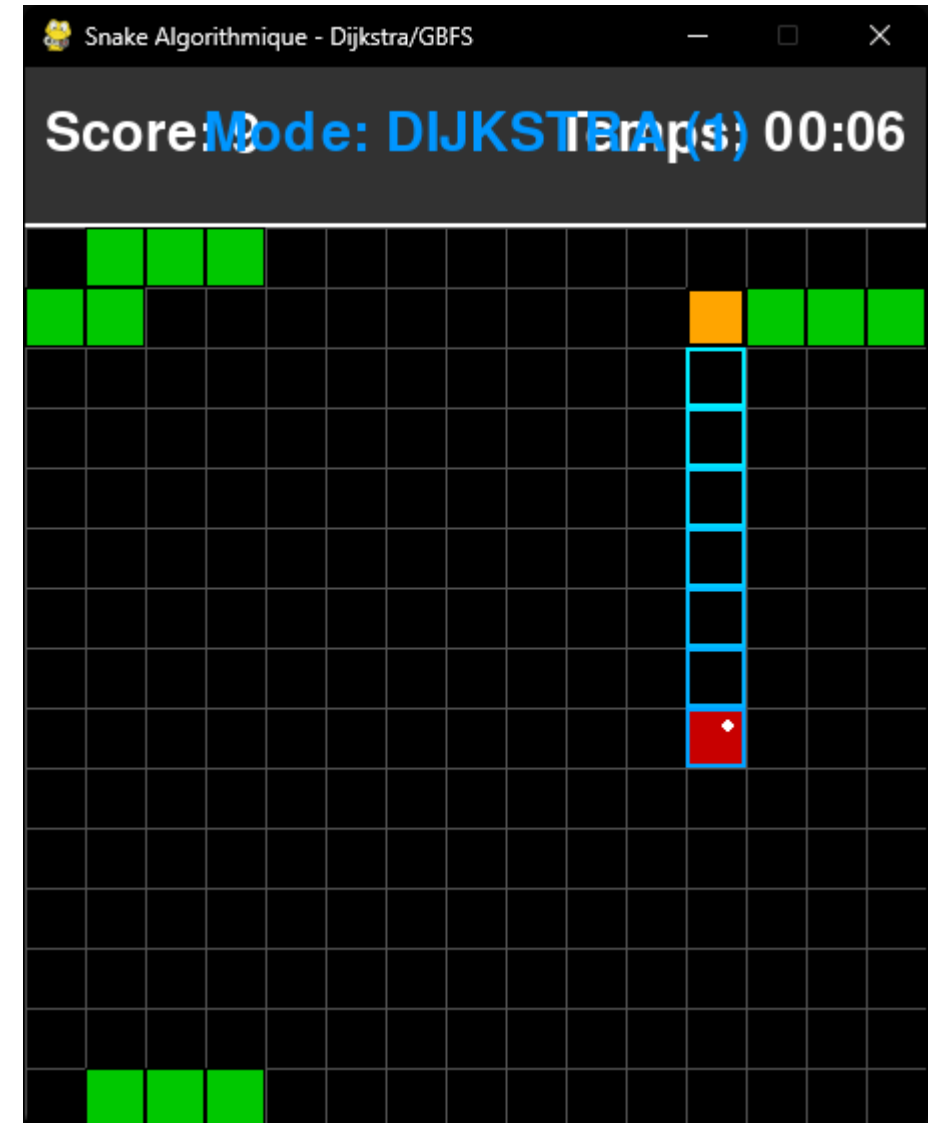
- Danger en face
- Danger à droite
- Danger à gauche

Direction

- Direction Gauche
- Direction Droite
- Direction Haut
- Direction Bas

Pomme

- Pomme Gauche
- Pomme Droite
- Pomme Haut
- Pomme Bas





Algos

Dijkstra & Greedy Best-First Search

Dijkstra

- **Objectif :** Trouver le chemin le **plus court** et le **moins coûteux** entre la tête du serpent et la pomme.
- **Comment il fonctionne (Simple) :**
 - Dijkstra explore le monde case par case, en se basant uniquement sur la **distance déjà parcourue** depuis le point de départ (la tête).
 - Il garantit qu'il ne passera jamais à côté d'une solution plus courte, car il explore toujours en priorité les cases qu'il a atteintes avec le coût le plus faible.
- **Analogie :** C'est comme un facteur qui connaît le coût exact (temps, énergie) pour aller à chaque maison. Il s'assure d'emprunter la route la plus économique, même si cela signifie explorer beaucoup de rues au début.
- **Avantages :**
 - **Optimalité :** Il trouve toujours le chemin le plus court (si un chemin existe).
 - **Dans le jeu :** Le serpent prendra le chemin le plus direct en nombre de cases.
- **Inconvénients :**
 - **Lent :** Il explore souvent de larges zones, même si elles sont dans la mauvaise direction, jusqu'à ce qu'il ait prouvé que ces chemins sont plus longs que les autres.

Greedy Best-First Search (GBFS)

Le "Chemin Le Plus Rapide Estimé"

- **Objectif :** Trouver un chemin **rapidement** vers la pomme, en se basant uniquement sur une **estimation** (heuristique) de la distance restante.
- **Comment il fonctionne (Simple) :**
 - GBFS ne se soucie pas de la distance déjà parcourue. Il choisit à chaque étape la case qui semble **la plus proche** de la cible (la pomme) en utilisant l'heuristique (ici, la distance de Manhattan).
 - Il est très "glouton" (*Greedy*) : il veut se rapprocher de la cible immédiatement.
- **Analogie :** C'est comme quelqu'un qui n'utilise que son GPS pour voir "à vol d'oiseau". Il se dirige toujours vers le point le plus proche sans se soucier des détours ou des obstacles possibles.
- **Avantages :**
 - **Rapide :** Il explore beaucoup moins de cases que Dijkstra et trouve souvent un chemin très vite.
 - **Dans le jeu :** Le serpent va directement vers la pomme.
- **Inconvénients :**
 - **Non-Optimalité :** Le chemin trouvé n'est pas garanti d'être le plus court. Il peut se faire piéger par des obstacles et prendre des détours inutiles s'il se concentre trop sur la cible immédiate.

Le votre qui sera meilleur que les deux combinés

- **Soyez inventif**



Ce que je vous demande

La soirée

Votre soirée

- **Développer en mode Reinforcement Learning**
 - Venir me donner votre pseudo github (un par groupe)
- ~~**Développer en mode Algorithmique (La semaine prochaine)**~~
- **Préparer la timeline // Le changelog**
 - Quand avez-vous commencer, quoi ?
 - Qui a fait le meilleur score ? Qui a fini le jeu ? En combien de temps ? (La courbe ?)
 - Pour des raisons d'équilibre :
 - Ne pas toucher à la clock
 - Ne pas toucher à la dimension de la grille
 - Ne pas changer le scoring
- **Préparer le tour de table**
 - Difficultés techniques et débogage
 - Obtention et validation des résultats
 - Transfert de compétences
- **Questionnaire forms**



Tour de table

Questions & Réponses

Thème #1 : Difficultés techniques et débogage (Code et Algorithme)

- Plop

Thème #2 : Obtention et validation des résultats (Métriques)

- Plop

Thème #3 : Mise en pratique en entreprise (Transfert de Compétences)

- Plop